

Credit Card Fraud Detection

Machine Learning Project Report

Contents

1	Part 1: Problem Definition	3
1.1	Python Code	3
1.2	Target Variable Distribution	3
1.3	Dataset Overview	3
1.4	Feature Descriptions	3
1.5	Data Types and Missing Values	4
2	Part 2: Advanced Data Exploration	5
2.1	Python Code	5
2.2	Skewness Analysis	5
2.3	Kurtosis Analysis	5
2.4	Correlation with Fraud	5
2.5	Outlier Detection (IQR Method)	6
2.6	Class Imbalance	6
2.7	Visualizations Generated	6
3	Part 3: Data Preprocessing	7
3.1	Step 1: Missing Values and Duplicates	7
3.2	Step 2: Dropping Irrelevant Columns	7
3.3	Step 3: Feature Engineering from Dates	7
3.4	Step 4: Outlier Capping	7
3.5	Step 5: Text Cleaning and Label Encoding	7
3.6	Step 6: Feature Scaling	8
3.7	Preprocessing Result	8
4	Part 4: Model Building	9
4.1	Model 1: KNN	9
4.2	Model 2: SVM	9

4.3	Model 3: Random Forest	9
4.4	Model Comparison	9
5	Part 5: Hyperparameter Tuning	10
5.1	Tuning KNN	10
5.2	Tuning SVM	10
5.3	Tuning Random Forest	10
5.4	Before vs After Tuning	10
6	Part 6: Evaluation	12
6.1	KNN Evaluation	12
6.2	SVM Evaluation	12
6.3	Random Forest Evaluation	12
6.4	Overfitting Analysis	12
7	Part 7: Improvement and Experimentation	14
7.1	Experiment 1: Before vs After Scaling	14
7.2	Experiment 2: Before vs After Tuning	14
7.3	Experiment 3: SMOTE Oversampling	14
7.4	Final Summary	15
7.5	Key Lessons Learned	15

1 Part 1: Problem Definition

This project focuses on detecting fraudulent credit card transactions using Machine Learning. Financial fraud causes major losses for banks and customers. This is a **Binary Classification Problem**:

- 0 = Legitimate Transaction
- 1 = Fraudulent Transaction

1.1 Python Code

```
1 import pandas as pd
2 import os
3
4 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
5 df_train = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTrain.csv"))
6 df_test = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTest.csv"))
```

1.2 Target Variable Distribution

Total Transactions : 1,296,675
Not Fraud (0) : 1,289,169 (99.42%)
Fraud (1) : 7,506 (0.58%)

The dataset is **highly imbalanced**. Fraud represents only 0.58% of all transactions. A model predicting “Not Fraud” for everything would achieve 99% accuracy but detect zero fraud.

1.3 Dataset Overview

Train Set: 1,296,675 rows x 23 columns
Test Set: 555,719 rows x 23 columns

1.4 Feature Descriptions

Feature	Description
trans_date_trans_time	Transaction date and time
cc_num	Credit card number (identifier)
merchant	Merchant name
category	Transaction category (e.g., grocery, gas)

amt	Transaction amount in USD
first, last	Cardholder name
gender	Cardholder gender (M/F)
street, city, state, zip	Cardholder address
lat, long	Customer latitude/longitude
city_pop	City population
job	Cardholder occupation
dob	Date of birth
trans_num	Unique transaction ID
unix_time	Transaction time (unix format)
merch_lat, merch_long	Merchant latitude/longitude
is_fraud	Target variable (0 or 1)

1.5 Data Types and Missing Values

- Numerical features: amt, lat, long, city_pop, merch_lat, merch_long, etc.
- Categorical features: merchant, category, gender, city, state, job, etc.
- Missing values: **None found**

2 Part 2: Advanced Data Exploration

This section performs deeper Exploratory Data Analysis (EDA) to understand statistical properties, detect outliers, and discover patterns.

2.1 Python Code

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 numerical_cols = ["amt", "lat", "long", "city_pop",
7                  "merch_lat", "merch_long"]
8 print(df[numerical_cols].describe())
9 print(df[numerical_cols].skew())
10 print(df[numerical_cols].kurtosis())
```

2.2 Skewness Analysis

```
amt      : 42.28 (Highly right-skewed)
city_pop : 5.59  (Right-skewed)
lat      : -0.19 (Approximately normal)
```

High skewness in `amt` means most transactions are small but a few are extremely large.

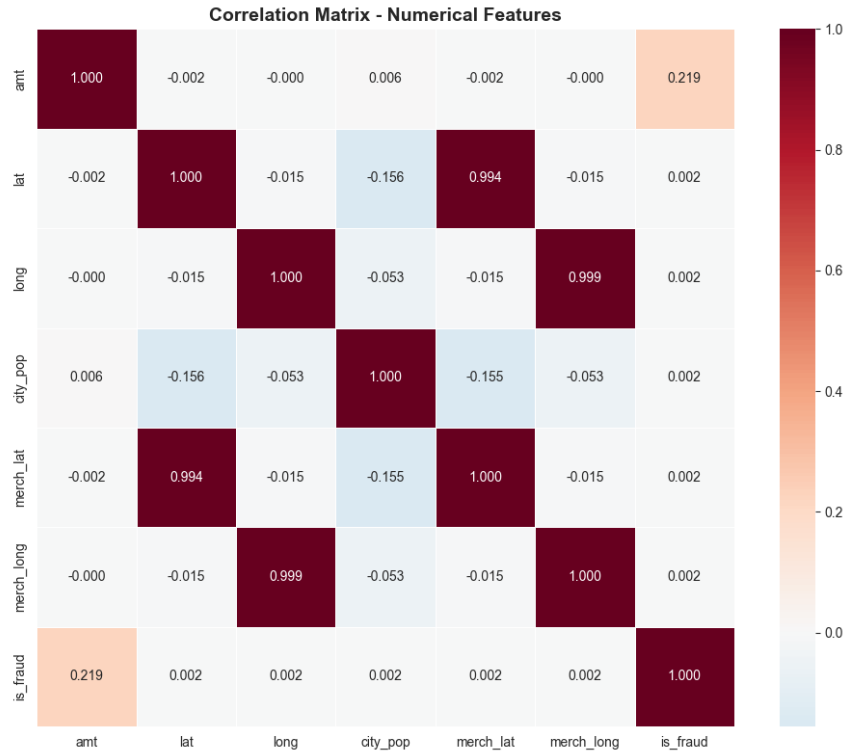
2.3 Kurtosis Analysis

```
amt      : 4545.65 (Heavy-tailed, extreme outliers)
city_pop : 37.61  (Heavy-tailed)
```

2.4 Correlation with Fraud

```
amt      : 0.2194 (Moderate - strongest predictor)
city_pop : 0.0021 (Weak)
lat      : 0.0019 (Weak)
```

Higher transaction amounts are more likely to be fraudulent.



2.5 Outlier Detection (IQR Method)

$$IQR = Q3 - Q1, \quad \text{Outlier if } x < Q1 - 1.5 \times IQR \text{ or } x > Q3 + 1.5 \times IQR$$

amt: 67,290 outliers (5.19%)
city_pop: 242,674 outliers (18.72%)

2.6 Class Imbalance

Not Fraud: 99.42% | Fraud: 0.58%
Imbalance Ratio = 1:171

For every 1 fraud transaction there are 171 normal ones. This extreme imbalance requires special handling.

2.7 Visualizations Generated

- amount_distribution.png – Fraud transactions involve higher amounts
- top_fraud_categories.png – Categories with highest fraud frequency
- gender_fraud.png – Fraud distribution across genders
- fraud_amount_boxplot.png – Amount comparison: fraud vs legitimate
- fraud_by_hour.png – Fraud rate varies by hour (higher at night)

3 Part 3: Data Preprocessing

Preprocessing transforms raw data into a format suitable for ML models.

3.1 Step 1: Missing Values and Duplicates

```
1 missing = df.isnull().sum().sum() # Result: 0
2 duplicates = df.duplicated().sum() # Result: 0
```

No missing values or duplicates found.

3.2 Step 2: Dropping Irrelevant Columns

```
1 drop_cols = ["Unnamed: 0", "cc_num", "first", "last",
2             "street", "trans_num", "unix_time"]
3 df = df.drop(columns=drop_cols)
```

These columns are identifiers or redundant, not useful for prediction.

3.3 Step 3: Feature Engineering from Dates

```
1 dataset["trans_hour"] = dataset["trans_date_trans_time"].dt.hour
2 dataset["trans_day"] = dataset["trans_date_trans_time"].dt.dayofweek
3 dataset["trans_month"] = dataset["trans_date_trans_time"].dt.month
4 dataset["age"] = (trans_date - dob).dt.days // 365
```

Extracts hour, day, month, and age as new numerical features from date columns.

3.4 Step 4: Outlier Capping

```
1 Q1 = df["amt"].quantile(0.25)
2 Q3 = df["amt"].quantile(0.75)
3 IQR = Q3 - Q1
4 df["amt"] = df["amt"].clip(lower=Q1-1.5*IQR, upper=Q3+1.5*IQR)
```

Capping limits extreme values instead of removing rows, preserving data.

3.5 Step 5: Text Cleaning and Label Encoding

```
1 # Clean text
2 dataset[col] = dataset[col].str.strip().str.lower()
3
4 # Binary encoding for gender
```

```
5 dataset["gender"] = dataset["gender"].map({"M": 0, "F": 1})
6
7 # LabelEncoder for other categoricals
8 le = LabelEncoder()
9 df["category"] = le.fit_transform(df["category"])
```

merchant: 693 unique values | category: 14
city: 906 | state: 51 | job: 497

3.6 Step 6: Feature Scaling

```
1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train)
```

StandardScaler transforms features to mean=0 and std=1. Essential for KNN and SVM which rely on distance calculations.

3.7 Preprocessing Result

X_train: (1,296,675, 17) - 17 features after preprocessing
X_test: (555,719, 17)

4 Part 4: Model Building

Three models are trained on a stratified subsample of 30,000 rows (full data is too large for KNN/SVM).

4.1 Model 1: KNN

```
1 knn = KNeighborsClassifier(n_neighbors=5)
2 knn.fit(X_train_sample, y_train_sample)
```

KNN finds the 5 closest training samples and uses majority voting.

Accuracy: 0.9961 | Precision: 0.0000 | Recall: 0.0000
F1: 0.0000 | TN=9961 FP=0 FN=39 TP=0

4.2 Model 2: SVM

```
1 svm = SVC(kernel="rbf", random_state=42)
2 svm.fit(X_train_sample, y_train_sample)
```

SVM finds the optimal decision boundary. RBF kernel allows non-linear separation.

Accuracy: 0.9961 | Precision: 0.0000 | Recall: 0.0000
F1: 0.0000 | TN=9961 FP=0 FN=39 TP=0

4.3 Model 3: Random Forest

```
1 rf = RandomForestClassifier(n_estimators=100, random_state=42)
2 rf.fit(X_train_sample, y_train_sample)
```

Random Forest builds 100 decision trees and uses majority voting.

Accuracy: 0.9966 | Precision: 1.0000 | Recall: 0.1282
F1: 0.2273 | TN=9961 FP=0 FN=34 TP=5

4.4 Model Comparison

Model	Accuracy	Precision	Recall	F1
KNN	0.9961	0.0000	0.0000	0.0000
SVM	0.9961	0.0000	0.0000	0.0000
Random Forest	0.9966	1.0000	0.1282	0.2273

KNN and SVM failed to detect any fraud due to extreme class imbalance. Random Forest detected 5 out of 39 fraud cases. F1-Score is used instead of accuracy because accuracy is misleading with imbalanced data.

5 Part 5: Hyperparameter Tuning

5.1 Tuning KNN

```
1 knn_params = {"n_neighbors": [3, 5, 7, 9, 11]}
2 knn_grid = GridSearchCV(KNeighborsClassifier(), knn_params,
3                          scoring="f1", cv=3)
```

K=3 -> F1=0.0328 | K=5 -> 0.0000 | K=7 -> 0.0000

Best K = 3. Still very weak.

5.2 Tuning SVM

```
1 svm_params = {
2     "kernel": ["rbf", "linear"],
3     "C": [0.1, 1, 10],
4     "class_weight": ["balanced"] # Key for imbalance!
5 }
```

class_weight="balanced" adjusts weights inversely proportional to class frequency, making fraud 171x more important than non-fraud.

Best: kernel=rbf, C=10, class_weight=balanced

F1 = 0.1818 | TP=7 (previously 0)

5.3 Tuning Random Forest

```
1 rf_params = {
2     "n_estimators": [50, 100, 200],
3     "max_depth": [10, 20, None],
4     "class_weight": ["balanced"]
5 }
```

Best: n_estimators=200, max_depth=10, balanced

F1 = 0.1923 | TP=15 (previously 5)

5.4 Before vs After Tuning

Model	Before F1	After F1	TP Before	TP After
KNN	0.0000	0.0000	0	0

SVM	0.0000	0.1818	0	7
Random Forest	0.2273	0.1923	5	15

6 Part 6: Evaluation

6.1 KNN Evaluation

Accuracy: 0.9959 | Precision: 0.0000

Recall: 0.0000 | F1-Score: 0.0000

TN=9959 FP=2 FN=39 TP=0

KNN completely failed to detect fraud.

6.2 SVM Evaluation

Accuracy: 0.9937 | Precision: 0.1842

Recall: 0.1795 | F1-Score: 0.1818

TN=9930 FP=31 FN=32 TP=7

SVM detected 7 fraud cases with 31 false alarms.

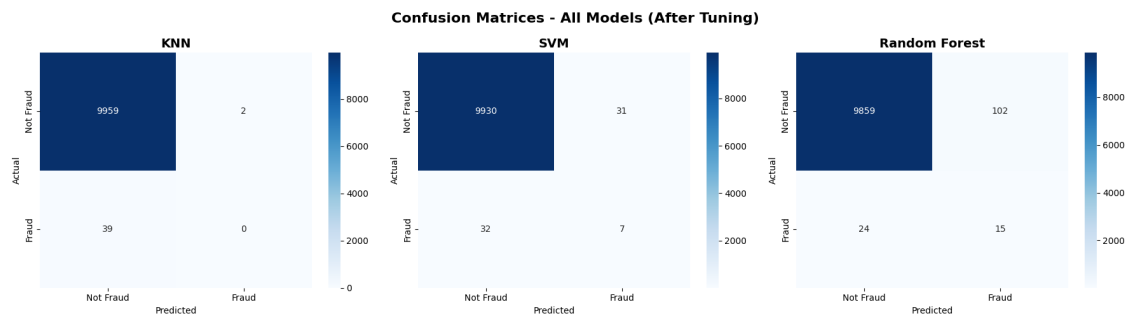
6.3 Random Forest Evaluation

Accuracy: 0.9874 | Precision: 0.1282

Recall: 0.3846 | F1-Score: 0.1923

TN=9859 FP=102 FN=24 TP=15

Random Forest detected 15 fraud cases (best recall) with 102 false alarms.

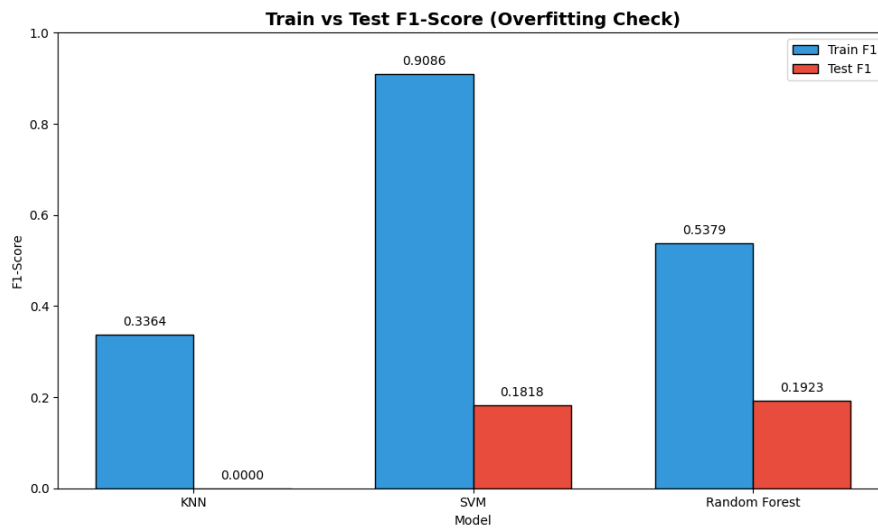


6.4 Overfitting Analysis

```
1 train_f1 = f1_score(y_train, model.predict(X_train))
2 test_f1 = f1_score(y_test, model.predict(X_test))
```

Model	Train F1	Test F1	Diagnosis
KNN	0.3364	0.0000	Overfitting
SVM	0.9086	0.1818	Overfitting
Random Forest	0.5379	0.1923	Overfitting

All models overfit because there are only 174 fraud samples in training data, making it hard to learn generalizable fraud patterns.



7 Part 7: Improvement and Experimentation

7.1 Experiment 1: Before vs After Scaling

```
1 # Train RF on unscaled vs scaled data
2 rf_unscaled = RandomForestClassifier(n_estimators=200,
3     max_depth=10, class_weight="balanced")
4 rf_unscaled.fit(X_train_unscaled, y_train)
```

	Before Scaling	After Scaling
Accuracy	0.9869	0.9874
Precision	0.1230	0.1282
Recall	0.3846	0.3846
F1-Score	0.1863	0.1923
Fraud Detected	15	15

Scaling has minimal impact on Random Forest (tree-based) but is critical for KNN and SVM (distance-based).

7.2 Experiment 2: Before vs After Tuning

Model	Before F1	After F1	TP Before	TP After

KNN	0.0000	0.0000	0	0
SVM	0.0000	0.1818	0	7
Random Forest	0.2273	0.1923	5	15

`class_weight="balanced"` was the most impactful improvement.

7.3 Experiment 3: SMOTE Oversampling

```
1 from imblearn.over_sampling import SMOTE
2 smote = SMOTE(random_state=42)
3 X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

SMOTE creates synthetic fraud samples by interpolating between existing ones:

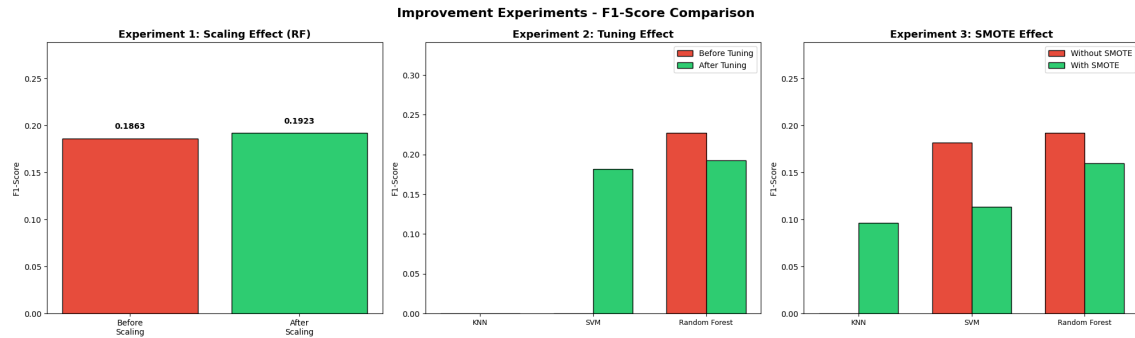
Before SMOTE: 29,826 Not Fraud, 174 Fraud

After SMOTE: 29,826 Not Fraud, 29,826 Fraud

Model	Without SMOTE F1	With SMOTE F1	SMOTE TP

KNN	0.0000	0.0963	9
SVM	0.1818	0.1136	5
Random Forest	0.1923	0.1600	20

SMOTE helped KNN detect 9 fraud (previously 0) and RF detect 20 fraud (highest count).



7.4 Final Summary

Stage	Action	Result
Part 4	Base models	KNN: TP=0, SVM: TP=0, RF: TP=5
Part 5	Tuning + class_weight	KNN: TP=0, SVM: TP=7, RF: TP=15
Part 7	SMOTE oversampling	KNN: TP=9, SVM: TP=5, RF: TP=20

Best Model: Random Forest (Tuned) – F1=0.1923, detects 15/39 fraud cases.

7.5 Key Lessons Learned

1. Accuracy is misleading with imbalanced data.
2. `class_weight="balanced"` is essential for rare class detection.
3. SMOTE helps models without built-in class weighting (KNN).
4. F1-Score is the correct metric for fraud detection.
5. Scaling is important for distance-based models (KNN, SVM).
6. Random Forest handles imbalanced data best among tested models.
7. All models showed overfitting due to very few fraud training samples.